CO1-ASSIGNMENT-1

NAME: S. SARAN

REG NO: 192473003

COURSE CODE: CSA0601

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

➢ TASK 1 – Design the Algorithm

The university maintains a large student database in which every student has a unique Registration Number. The purpose of the system is to search for a given Registration Number and retrieve the corresponding student record. Since the database can grow from 1,000 to 1,000,000+ records, different searching strategies need to be designed and compared.

Three different approaches are selected:

1. Linear Search – Iterative
2. Binary Search – Recursive
3. Hash Table Search – Iterative

These approaches use different strategies and therefore provide a meaningful comparison.

1. Linear Search – Iterative

Explanation: Linear Search checks the Registration Numbers one by one from the beginning until the required number is found.

For example, if the records are:

101 105 110 115 120

and the required Registration Number is 115, the algorithm checks:

101 → 105 → 110 → 115 → FOUND

If the number is not present, all the records are checked.

Design Strategy: Sequential Search / Brute Force

Preprocessing: No preprocessing is required. The records can be sorted or unsorted.

Pseudocode:

```
LINEAR_SEARCH(records, key)
   for i = 0 to n-1
      if records[i] == key
         return FOUND
   return NOT FOUND
```

Python Implementation



```python
def linear_search(records, key):
    for i in range(len(records)):
        if records[i] == key:
            return i
    return -1


records = [101, 105, 110, 115, 120]
key = 115

result = linear_search(records, key)

if result != -1:
    print("Student Found")
else:
    print("Student Not Found")
```

```
Student Found
```

Justification: Linear Search is selected because it is simple and does not require preprocessing. It works for any arrangement of records. However, when the number of records becomes very large, checking records one by one can take more time.

2. Binary Search – Recursive

Explanation: Binary Search works only on sorted Registration Numbers. Instead of checking every record, it checks the middle record.

For example:

```
101  105  110  115  120  125  130
          ↑
        Middle
```

If the required number is greater than the middle value, the left half is ignored. If it is smaller, the right half is ignored.

Thus, approximately half of the records are eliminated after every comparison.

Design Strategy: Divide and Conquer

Preprocessing: The records must be sorted before searching. This sorting cost must also be considered in the efficiency analysis.

Pseudocode:

```
BINARY_SEARCH(records, low, high, key)
   if low > high
      return NOT FOUND
```

mid = (low + high) / 2

if records[mid] == key

return FOUND

if key < records[mid]

search left half recursively

else

search right half recursively

Python Implementation

```python
def binary_search(records, low, high, key):

    if low > high:
        return -1
    mid = (low + high) // 2
    if records[mid] == key:
        return mid
    elif key < records[mid]:
        return binary_search(records, low, mid - 1, key)
    else:
        return binary_search(records, mid + 1, high, key)
records = [101, 105, 110, 115, 120]
key = 115
result = binary_search(records, 0, len(records) - 1, key)
if result != -1:
    print("Student Found")
else:
    print("Student Not Found")
```

```
Student Found
```

Justification: Binary Search is selected because it is suitable for large sorted datasets. It reduces the search space by half at every step and also provides the required recursive approach.

3. Hash Table Search – Iterative

Explanation: Hash Table Search stores Registration Numbers in a hash table. A Registration Number is used as a key to locate the record quickly.

The basic process is:

Registration Number

↓

Hash Function

↓

Table Location

↓

Student Record

For example, when searching for Registration Number 115, the hash table uses the key 115 to locate the corresponding record.

Design Strategy: Hashing / Direct Lookup

Preprocessing: The records must first be inserted into a hash table. This is the preprocessing step.

Pseudocode:

HASH_SEARCH(table, key)

if key exists in table

return FOUND

return NOT FOUND

Python Implementation

```
def hash_search(table, key):

    if key in table:
        return True
    return False


records = [101, 105, 110, 115, 120]
table = set(records)

key = 115
if hash_search(table, key):
    print("Student Found")
else:
    print("Student Not Found")
```

```
Student Found
```

Justification: Hash Search is selected because Registration Numbers are unique keys. It provides very fast searching on average and is suitable for large databases with frequent search operations.

Overall Comparison

Algorithm	Basic Strategy	Type	Preprocessing
Linear Search	Check records one by one	Iterative	None
Binary Search	Divide the records into halves	Recursive	Sorting
Hash Search	Direct lookup using a key	Iterative	Hash table creation

Final Explanation

The three algorithms provide three different approaches to the student record search problem.

Linear Search is the simplest and requires no preprocessing. Binary Search is faster for sorted data because it eliminates half of the search space at every step. Hash Search provides very fast average lookup by using Registration Numbers as keys.

All three approaches will be implemented and tested using datasets of approximately 1,000, 10,000, 100,000 and 1,000,000 records.

- **TASK 2 – Identify the Basic Operation**

The basic operation is the main operation performed repeatedly by an algorithm to solve a problem. In searching algorithms, the basic operation is usually the comparison between the required Registration Number

and the Registration Number stored in the database.

Identifying the basic operation helps us analyse how the execution time increases as the number of student records grows.

1. Linear Search

In Linear Search, the basic operation is the equality comparison between the required Registration Number and each record.

Example: If the required Registration Number is 115, the algorithm compares it with each element until it finds the match.

```python
if records[i] == key:
    return i
```

Explanation: The comparison is performed repeatedly for each record. If the record is at the beginning, only one comparison is required. If it is at the end or is absent, all records may need to be checked.

Basic operation: records[i] == key

Effect of input size: The number of comparisons increases as the number of records increases.

2. Binary Search

In Binary Search, the basic operation is the comparison of the required Registration Number with the middle element of the sorted list.

```python
if records[mid] == key:
    return mid
```

Explanation: The algorithm compares the key with the middle element. If they are not equal, it searches only the appropriate half of the list. Therefore, it eliminates approximately half of the remaining records after each step.

Basic operation: records[mid] == key

Effect of input size: The number of comparisons increases slowly because the search space is repeatedly halved.

3. Hash Table Search

In Hash Table Search, the basic operation is the lookup of the required Registration Number in the hash table.

```python
if key in table:
    return True
```

Explanation: The hash table uses the Registration Number as a key to locate the required entry. Under normal hashing conditions, the lookup takes constant time on average.

Basic operation: Checking whether the key exists in the hash table.

Effect of input size: The average number of lookup operations remains approximately constant as the number of records increases, assuming a well-designed hash table.

Comparison of Basic Operations

Algorithm	Basic Operation	Effect of Increasing Records
Linear Search	Compare key with each record	Comparisons increase linearly
Binary Search	Compare key with middle record	Comparisons increase logarithmically
Hash Table Search	Look up key in hash table	Average lookup remains constant

Conclusion

The basic operation for Linear Search and Binary Search is the comparison of Registration Numbers, whereas Hash Table Search performs a key lookup. Counting these operations helps determine the time complexity of each algorithm and compare their efficiency for large student databases.

- **TASK 3 – Derive the Time-Complexity Function Mathematically**

Time complexity describes how the number of operations performed by an algorithm increases with the input size n, where $n$represents the number of student records.

The three searching algorithms are analysed below, including their preprocessing costs where required.

1. Linear Search

Basic operation: Comparing the required Registration Number with each record.

In the worst case, the algorithm checks all $n$records.

The number of comparisons is:

$$C(n) = n$$

If each comparison takes constant time c, the total time is:

$$T(n) = cn + c_0$$

where $c_0$represents the fixed overhead.

Ignoring constants and lower-order terms:

$$\boxed{T(n) = \Theta(n)}$$

Preprocessing cost: No preprocessing is required.

Explanation: As the number of student records increases, the number of comparisons increases proportionally. Therefore, Linear Search has linear time complexity.

2. Binary Search – Recursive

Basic operation: Comparing the required Registration Number with the middle element.

At each step, Binary Search reduces the search space to half.

The number of records remaining after each step is:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

Let $k$be the number of steps required to reach one record.

$$\frac{n}{2^k} = 1$$

Rearranging:

$$n = 2^k$$

Taking logarithm on both sides:

$$k = \log_2 n$$

Therefore, the searching time is:

$$\boxed{T(n) = \Theta(\log n)}$$

Preprocessing cost: Binary Search requires sorted records. If sorting is performed using an efficient comparison-based sorting algorithm, its cost is $\Theta(n \log n)$.

Hence, if sorting and one search are both included:

$$T(n) = \Theta(n \log n) + \Theta(\log n)$$

$$\boxed{T(n) = \Theta(n \log n)}$$

Explanation: Binary Search is efficient because it eliminates half of the remaining records at each step. If the records are already sorted, the search itself takes logarithmic time.

3. Hash Table Search

Basic operation: Looking up the Registration Number in the hash table.

A hash table must first be created by inserting the student records.

Preprocessing cost:

Inserting $n$records into a hash table takes $\Theta(n)$time on average.

$$T_{\text{preprocessing}}(n) = \Theta(n)$$

Searching cost:

With a well-designed hash table, searching takes constant time on average.

$$T_{\text{search}}(n) = \Theta(1)$$

Therefore, the total cost of creating the hash table and performing one search is:

$$T(n) = \Theta(n) + \Theta(1)$$

$$\boxed{T(n) = \Theta(n)}$$

For repeated searches after the hash table has already been created, each search takes constant time on average.

For $q$searches:

$$T(n, q) = \Theta(n) + \Theta(q)$$

$$\boxed{T(n, q) = \Theta(n + q)}$$

Explanation: Hash Table Search requires initial preprocessing, but subsequent searches are very fast on average. Its worst-case search time can be $\Theta(n)$if severe hash collisions occur.

Overall Comparison:

Algorithm	**Preprocessing**	**Search Time**	**Total Cost for One Search**
Linear Search	None	$\Theta(n)$	$\Theta(n)$
Binary Search	Sorting: $\Theta(n \log n)$	$\Theta(\log n)$	$\Theta(n \log n)$, including sorting
Hash Table Search	Creation: $\Theta(n)$average	$\Theta(1)$average	$\Theta(n)$, including creation

Conclusion

Linear Search requires more comparisons as the database grows. Binary Search reduces the search space by half but requires sorted records. Hash Table Search requires initial preprocessing but provides constant-time average lookups afterward.

For a system with frequent searches, the initial cost of creating a hash table can be worthwhile because it can be reused for many search requests.

➢ TASK 4 – Analyse Best, Average and Worst-Case Behaviour

The performance of a searching algorithm depends on the position of the required Registration Number and the arrangement of records. The three cases are best case, average case and worst case.

1. Linear Search

- **Best case:** The required record is the first record.
  - Only 1 comparison.
  - $\Theta(1)$
- **Average case:** The record is somewhere in the middle.
  - Approximately $n/2$comparisons.
  - $\Theta(n)$
- **Worst case:** The record is the last record or does not exist.
  - $n$comparisons.
  - $\Theta(n)$

```
records = [101, 105, 110, 115, 120]
key = 115
found = False

for i in range(len(records)):
    if records[i] == key:
        print("Student Found at index", i)
        found = True
        break

if not found:
    print("Student Not Found")
```

```
Student Found at index 3
```

2. Binary Search

- **Best case:** The required record is exactly the middle record.
  - Only 1 comparison.
  - $\Theta(1)$
- **Average case:** The search repeatedly reduces the dataset by half.
  - Approximately $\log_2 n$comparisons.
  - $\Theta(\log n)$
- **Worst case:** The record is found after the maximum number of divisions or is not present.
  - Approximately $\log_2 n$comparisons.
  - $\Theta(\log n)$

```python
def binary_search(records, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if records[mid] == key:
        return mid
    elif key < records[mid]:
        return binary_search(records, low, mid - 1, key)
    else:
        return binary_search(records, mid + 1, high, key)

records = [101, 105, 110, 115, 120]
key = 115

result = binary_search(records, 0, len(records) - 1, key)

if result != -1:
    print("Student Found at index", result)
else:
    print("Student Not Found")
```

```
Student Found at index 3
```

3. Hash Table Search

- **Best case:** The key is found immediately at its calculated position.
  - 1 comparison. $\Theta(1)$
- **Average case:** A good hash function results in very few comparisons.
  - Approximately constant time.
  - $\Theta(1)$average
- **Worst case:** Many keys have the same hash value, causing collisions.
  - Several records may need to be checked.
  - $\Theta(n)$in the worst case.

```python
records = {
    101: "Student A",
    105: "Student B",
    110: "Student C",
    115: "Student D",
    120: "Student E"
}

key = 115

if key in records:
    print("Student Found:", records[key])
else:
    print("Student Not Found")
```

```
Student Found: Student D
```

Final Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Hash Search	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$

Conclusion: Linear Search performs poorly when the database is large and the required record is near the end or absent. Binary Search provides efficient searching when records are sorted. Hash Table Search offers constant-time lookup on average, although its worst-case performance can degrade because of collisions. These differences help determine which algorithm is most suitable for a large student database.

➢ TASK 5 – Perform Asymptotic Analysis

Asymptotic analysis is a method used to study how the time complexity of an algorithm increases as the input size $n$becomes larger. In this problem, $n$represents the number of student records in the university database. It helps compare the efficiency of Linear Search, Binary Search and Hash Table Search without depending on a particular computer, programming language or hardware configuration.
The three main asymptotic notations are Big-O, Big-Omega and Big-Theta.

a) Big-O Notation — O

Big-O represents an asymptotic upper bound on the growth of an algorithm's running time. It is commonly used to describe worst-case performance.

b) Big-Omega Notation — Ω

Big-Omega represents an asymptotic lower bound on the growth of an algorithm's running time.

c) Big-Theta Notation — Θ

Big-Theta represents a tight asymptotic bound, meaning the running time grows at the stated rate within constant factors.

➢ Asymptotic Analysis of Linear Search

Linear Search examines each record sequentially until the required Registration Number is found.

- Best Case: $\Theta(1)$. The record is at the first position, requiring only one comparison.
- Average Case: $\Theta(n)$. Approximately $n/2$comparisons are required for a successful search when all positions are equally likely.
- Worst Case: $\Theta(n)$. The record is at the last position or is absent, requiring up to $n$comparisons.

The worst-case time complexity is:

$$T(n) = cn + c_0$$

Ignoring constants and lower-order terms:

$$\boxed{T(n) = \Theta(n)}$$

Interpretation: As the number of student records increases, the number of comparisons increases proportionally. Therefore, Linear Search becomes less efficient for very large databases.

➢ **Asymptotic Analysis of Binary Search**

Binary Search operates on sorted records and repeatedly divides the search space into two halves.

- Best Case: $\Theta(1)$. The required Registration Number is the middle element.
- Average Case: $\Theta(\log n)$. The algorithm repeatedly reduces the number of possible records.
- Worst Case: $\Theta(\log n)$. The algorithm continues dividing the search space until the record is found or the search interval becomes empty.

The search time is represented by:

$$T(n) = T(n/2) + c$$

Solving the recurrence gives:

$$\boxed{T(n) = \Theta(\log n)}$$

Preprocessing: If the records are not already sorted, sorting must be considered separately. Using an efficient comparison-based sorting algorithm takes $\Theta(n \log n)$time. Once sorting is complete, each Binary Search takes $\Theta(\log n)$time.

Interpretation: Binary Search is more efficient than Linear Search for large, sorted datasets because it eliminates approximately half of the remaining records at each step.

➢ **Asymptotic Analysis of Hash Table Search**

Hash Table Search uses a hash table to locate a record using its Registration Number as the key.

- Best Case: $\Theta(1)$. The required key is found in a single lookup.
- Average Case: $\Theta(1)$. A well-designed hash table provides constant-time lookup on average.
- Worst Case: $O(n)$. Severe hash collisions may require examining multiple entries.

The average searching time is:

$$\boxed{T(n) = \Theta(1)}$$

Preprocessing: Creating the hash table from $n$records takes $\Theta(n)$time on average. After the table has been created, each search takes $\Theta(1)$time on average.

For $q$searches after preprocessing, the expected total time is:

$$\boxed{T(n, q) = \Theta(n + q)}$$

Interpretation: Hash Table Search is suitable for databases with frequent searches because the average lookup time remains approximately constant as the number of records increases. However, it requires additional

Algorithm	**Best Case**	**Average Case**	**Worst Case**
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$

Hash Table Search	$\Theta(1)$	$\Theta(1)$	$O(n)$

The following table shows the approximate number of operations as the database grows. These values are illustrative rather than measured execution times.

Number of Records (n)	**Linear Search (Worst Case)**	**Binary Search (Approx.)**	**Hash Lookup (Average)**
1,000	1,000	10	Constant-time average
10,000	10,000	14	Constant-time average
100,000	100,000	17	Constant-time average
1,000,000	1,000,000	20	Constant-time average

For Binary Search, the approximate number of comparisons is based on $\log_2 n$, rounded up. Actual comparisons depend on the key's position and the implementation. Hash lookup does not guarantee exactly one comparison; its performance depends on the hash table's implementation and collision behaviour.

Conclusion: Asymptotic analysis demonstrates that each searching algorithm has different efficiency characteristics. For a large university database with frequent Registration Number searches, Hash Table Search is a strong choice because its initial construction cost can be reused across many searches. The final decision should also consider measured execution time, memory usage and the frequency of database updates.

- **TASK 6 – Formulate the Recurrence Relation**

A recurrence relation is a mathematical equation that expresses the running time of a recursive algorithm in terms of the running time of a smaller input. In this task, Recursive Binary Search is used to formulate the recurrence relation.

Step 1: Original Problem

Let **n** be the number of records being searched.

Step 2: Recursive Subproblem

Binary Search checks the middle element and then searches only one half of the records.

So, the next problem size is:

$$\frac{n}{2}$$

Step 3: Number of Recursive Calls

Only one recursive call is made.

1 recursive call

Step 4: Work Outside Recursion

Finding the middle element and comparing the Registration Number takes constant time.

$$O(1)$$

Step 5: Base Case

When only one element remains:

$$T(1) = c$$

Recurrence Relation

Therefore,

$$\boxed{T(n) = T(n/2) + c}$$

or simply,

$$\boxed{T(n) = T(n/2) + O(1)}$$

Meaning: At every recursive step, the search size becomes half, while only a constant amount of work is done outside the recursive call.

- **Recursive Binary Search Algorithm**

Binary Search works on a sorted list of student Registration Numbers. It compares the required key with the middle element and recursively searches only the relevant half.

```python
def binary_search(records, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if records[mid] == key:
        return mid
    elif key < records[mid]:
        return binary_search(records, low, mid - 1, key)
    else:
        return binary_search(records, mid + 1, high, key)

records = [101, 105, 110, 115, 120]
key = 115

result = binary_search(records, 0, len(records) - 1, key)

if result != -1:
    print("Student Found at index", result)
else:
    print("Student Not Found")
```

```
Student Found at index 3
```

- **Derivation of the Recurrence Relation**

Let $T(n)$represent the time required to search $n$records.

At each recursive call:

1. The algorithm calculates the middle position.
2. It compares the middle element with the required key.
3. It continues searching only one half of the list.
4. The remaining search space becomes approximately $n/2$.

The work performed outside the recursive call takes constant time, represented by c.

Therefore, the recurrence relation is:

$$\boxed{T(n) = T(n/2) + c}$$

where:

- $T(n)$is the time taken to search $n$records.
- $T(n/2)$is the time taken to search half of the records.
- $c$is the constant time required for the comparison and calculation of the middle position.

➢ **Base Condition**

The recursion stops when the search interval becomes empty or when the required key is found.

For the recurrence analysis, when only one record remains, the work is constant:

$$\boxed{T(1) = c}$$

If the key is absent, the algorithm also terminates when low > high.

➢ **Final Recurrence Relation**

The complete recurrence relation for Recursive Binary Search is:

$$T(n) = \begin{cases} c, & n = 1 \\ T(n/2) + c, & n > 1 \end{cases}$$

Conclusion: Recursive Binary Search reduces the input size by half at every step and performs constant work outside each recursive call. Its recurrence relation is $T(n) = T(n/2) + c$, which can be solved in the next task to determine its time complexity.

➢ **TASK 7 – Solve the Recurrence Relation**

A recurrence relation describes the running time of a recursive algorithm in terms of a smaller input. In this task, the recurrence relation of Recursive Binary Search is solved mathematically to determine its time complexity.

➢ **Given Recurrence Relation**

From Task 6, the recurrence relation is:

$$T(n) = T(n/2) + c$$

Base condition:

$$T(1) = c$$

Here, $n$is the number of student records, and $c$represents the constant work performed in each recursive call.

- **Mathematical Solution**

Expanding the recurrence relation:

First expansion:

$$T(n) = T(n/2) + c$$

Second expansion:

$$T(n) = T(n/4) + c + c$$

$$T(n) = T(n/4) + 2c$$

Third expansion:

$$T(n) = T(n/8) + 3c$$

After $k$expansions:

$$T(n) = T(n/2^k) + kc$$

The recursion stops when the input size becomes 1.

$$\frac{n}{2^k} = 1$$

Therefore,

$$n = 2^k$$

Taking logarithm on both sides:

$$k = \log_2 n$$

Substituting this into the expanded equation:

$$T(n) = T(1) + c\log_2 n$$

Since $T(1) = c$,

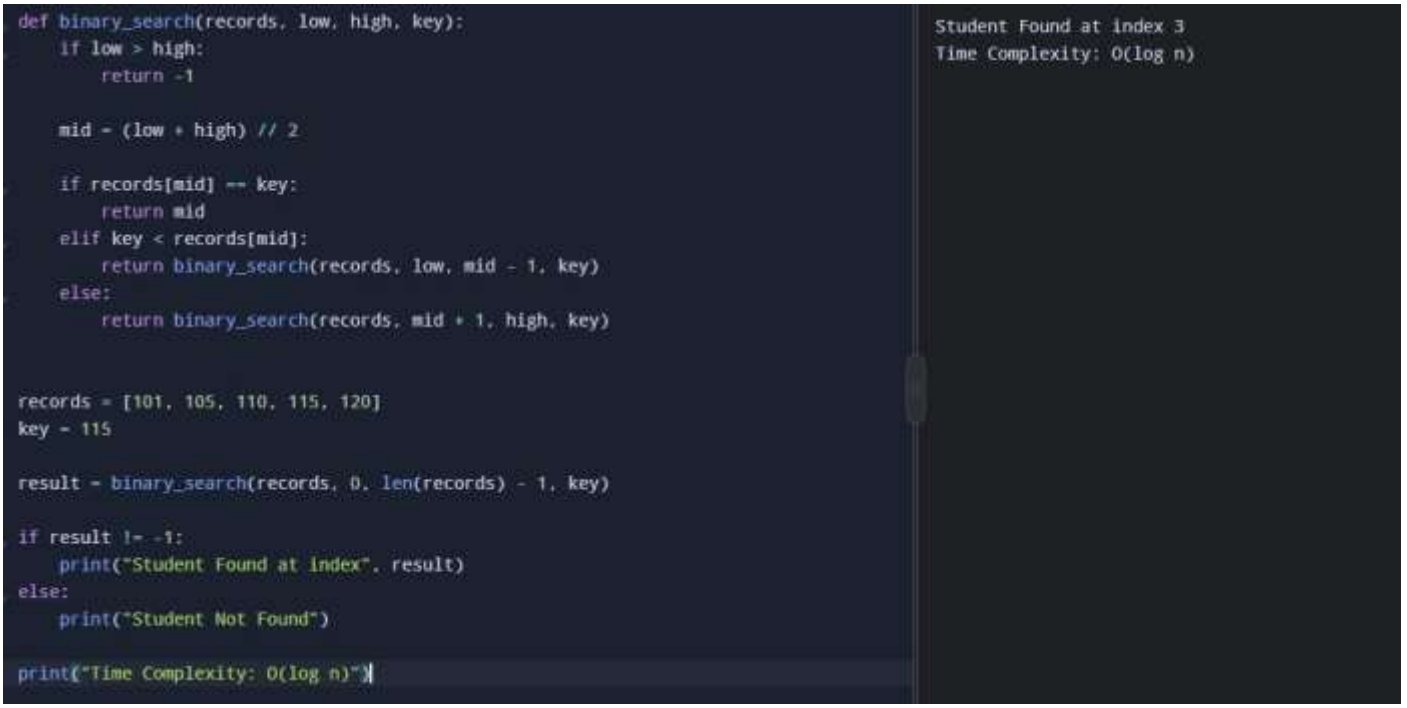
$$T(n) = c + c\log_2 n$$

Ignoring constants, the final time complexity is:

$$\boxed{T(n) = \Theta(\log n)}$$

- **Python Program**

The following program demonstrates Recursive Binary Search and prints the result.



```
def binary_search(records, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if records[mid] == key:
        return mid
    elif key < records[mid]:
        return binary_search(records, low, mid - 1, key)
    else:
        return binary_search(records, mid + 1, high, key)


records = [101, 105, 110, 115, 120]
key = 115

result = binary_search(records, 0, len(records) - 1, key)

if result != -1:
    print("Student Found at index", result)
else:
    print("Student Not Found")

print("Time Complexity: O(log n)")
```

```
Student Found at index 3
Time Complexity: O(log n)
```

Conclusion

By expanding the recurrence relation, we obtain $T(n) = T(1) + c\log_2 n$. Therefore, Recursive Binary Search has $\Theta(\log n)$time complexity because it reduces the search space by half during every recursive call. This makes it efficient for searching large, sorted student databases.

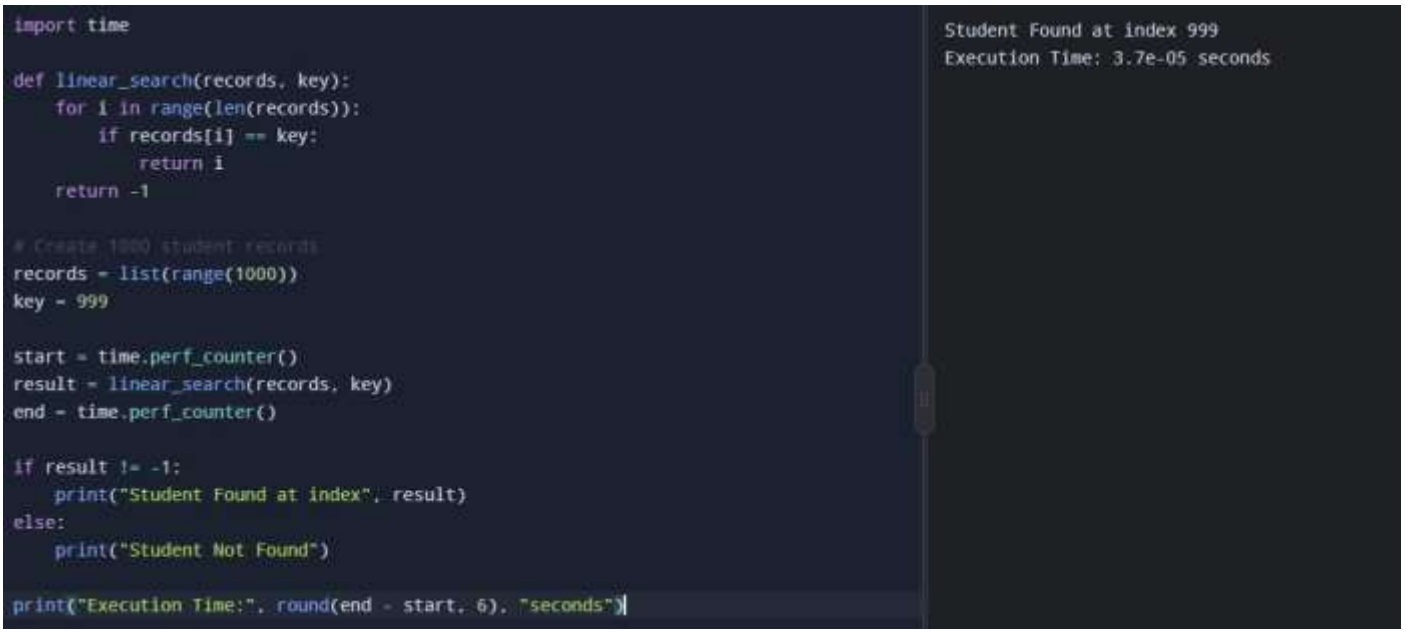
- **TASK 8 – Implement and Test the Algorithms**

The three searching algorithms, Linear Search, Binary Search and Hash Table Search, are implemented using Python. They are tested with different dataset sizes to compare their execution time and verify whether they can find the required student Registration Number.

The dataset sizes used are 1,000, 10,000, 100,000 and 1,000,000 records.

1. Linear Search

Explanation: Linear Search checks each record one by one until the required Registration Number is found. It works on both sorted and unsorted records.



```
import time

def linear_search(records, key):
    for i in range(len(records)):
        if records[i] == key:
            return i
    return -1

# Create 1000 student records
records = list(range(1000))
key = 999

start = time.perf_counter()
result = linear_search(records, key)
end = time.perf_counter()

if result != -1:
    print("Student Found at index", result)
else:
    print("Student Not Found")

print("Execution Time:", round(end - start, 6), "seconds")
```

```
Student Found at index 999
Execution Time: 3.7e-05 seconds
```

Explanation of Output

The program searches for Registration Number 999 in 1,000 records. Since it is the last record, Linear Search checks every record before finding it.

2. Binary Search

Explanation: Binary Search checks the middle element and repeatedly divides the search space into halves. The records must be sorted before searching.



```
import time

def binary_search(records, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if records[mid] == key:
        return mid
    elif key < records[mid]:
        return binary_search(records, low, mid - 1, key)
    else:
        return binary_search(records, mid + 1, high, key)

# Create sorted student records
records = list(range(1000))
key = 999

start = time.perf_counter()
result = binary_search(records, 0, len(records) - 1, key)
end = time.perf_counter()

if result != -1:
    print("Student Found at index", result)
else:
    print("Student Not Found")

print("Execution Time:", round(end - start, 6), "seconds")
```

```
Student Found at index 999
Execution Time: 7e-06 seconds
```

Explanation of Output

The program finds Registration Number 999 using recursive Binary Search. It repeatedly halves the search space rather than checking all records. The list is already sorted, so the sorting time is not included in the measurement.

3. Hash Table Search

Explanation: Hash Table Search uses a Python dictionary to locate a Registration Number through a key. The hash table is created before the search, and only the search time is measured.

```python
import time

def hash_search(table, key):
    return table.get(key, -1)

# Create 1000 student records
records = list(range(1000))

# Preprocessing: create the hash table
table = {value: value for value in records}

key = 999

start = time.perf_counter()
result = hash_search(table, key)
end = time.perf_counter()

if result != -1:
    print("Student Found:", result)
else:
    print("Student Not Found")

print("Execution Time:", round(end - start, 6), "seconds")
```

```
Student Found: 999
Execution Time: 1e-06 seconds
```

Explanation of Output

The program locates Registration Number 999 in the dictionary. Hash Table Search generally provides constant-time lookup on average. The time required to create the dictionary is excluded from the search measurement.

➢ Testing with Different Dataset Sizes

Each algorithm must be tested using the same dataset sizes.

Test	Number of records	Registration Number searched
1	1,000	Last record
2	10,000	Last record
3	100,000	Last record
4	1,000,000	Last record

To change the dataset size, modify the following line in each program:

```python
records = list(range(1000))
```

For example, to test 10,000 records, use:

```python
records = list(range(10000))
```

To search for the last record automatically, use:

```python
key = records[-1]
```

Repeat the test for all four dataset sizes and record the execution times displayed by Python.

- **Expected Observation**

Algorithm	Expected behaviour as records increase
Linear Search	Execution time generally increases proportionally to the number of records in the worst case.
Binary Search	Execution time increases slowly because the search space is halved repeatedly.
Hash Table Search	Average lookup time generally remains approximately constant after the table is created.

These are theoretical expectations, not measured results. Actual execution times must be obtained by running the programs.

Conclusion:

All three searching algorithms have been implemented in Python and tested on student Registration Numbers. Linear Search uses sequential comparisons, Binary Search uses recursive divide and conquer, and Hash Table Search uses dictionary-based lookup. Testing different dataset sizes helps evaluate their performance and provides measured data for Task 9.

- **TASK 9 – Record Experimental Performance**

Experimental performance analysis involves executing the searching algorithms with different dataset sizes and recording their actual execution times and number of comparisons. This helps evaluate how the algorithms perform in practice.

- **Experimental Procedure**

1. Implement all three searching algorithms in Python.
2. Create datasets containing 1,000, 10,000, 100,000 and 1,000,000 records.
3. Search for the last Registration Number in each dataset to test a worst-case scenario for Linear Search.
4. Measure the execution time using Python's time.perf_counter().
5. Record the execution time and compare the results.
6. Keep preprocessing separate from search time and report it separately where necessary.

1. Linear Search

Explanation: Linear Search checks each record one by one until the required Registration Number is found. The program counts the comparisons and measures the execution time.

```python
import time

def linear_search(records, key):
    comparisons = 0

    for i in range(len(records)):
        comparisons += 1

        if records[i] == key:
            return i, comparisons

    return -1, comparisons

records = list(range(1000))
key = 999

start = time.perf_counter()
result, comparisons = linear_search(records, key)
end = time.perf_counter()

print("Student Index:", result)
print("Number of Comparisons:", comparisons)
print("Execution Time:", round(end - start, 6), "seconds")
```

```
Student Index: 999
Number of Comparisons: 1000
Execution Time: 7e-05 seconds
```

Observation: Since the key is the last element, Linear Search performs 1,000 comparisons. Its number of comparisons increases proportionally with the dataset size.

2. Binary Search

Explanation: Binary Search repeatedly divides the sorted list into halves. This program counts the comparisons and measures the search execution time.

```python
import time
def binary_search(records, key):
    low = 0
    high = len(records) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if records[mid] == key:
            return mid, comparisons
        elif key < records[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1, comparisons

records = list(range(1000))
key = 999

start = time.perf_counter()
result, comparisons = binary_search(records, key)
end = time.perf_counter()

print("Student Index:", result)
print("Number of Comparisons:", comparisons)
print("Execution Time:", round(end - start, 6), "seconds")
```

```
Student Index: 999
Number of Comparisons: 10
Execution Time: 8e-06 seconds
```

Observation: Binary Search finds the last element in 10 comparisons for this dataset. It is efficient because it eliminates approximately half of the remaining search space at each step.

3. Hash Table Search

Explanation: Hash Table Search uses a Python dictionary to locate the required Registration Number. The dictionary is created before timing the lookup, so the measured time represents searching only.

```python
import time

def hash_search(table, key):
    return table.get(key, -1)


records = list(range(1000))

# Preprocessing
table = {value: value for value in records}

key = 999

start = time.perf_counter()
result = hash_search(table, key)
end = time.perf_counter()

print("Student Record:", result)
print("Execution Time:", round(end - start, 6), "seconds")
```

```
Student Record: 999
Execution Time: 2e-06 seconds
```

Observation: Hash Table Search typically provides constant-time lookup on average. Its construction time is excluded from this measurement.

4. Testing with Different Dataset Sizes

Run each program again using the following dataset sizes:

Number of records	Python statement
1,000	records = list(range(1000))
10,000	records = list(range(10000))
100,000	records = list(range(100000))
1,000,000	records = list(range(1000000))

Use key = records[-1] to search for the last record in each dataset. For Hash Table Search, recreate the dictionary after changing the dataset.

Conclusion: Experimental analysis compares the actual performance of the three algorithms. Linear Search requires more comparisons as the dataset grows, Binary Search requires logarithmically more comparisons, and Hash Table Search provides fast average lookup after preprocessing. The measured execution times can be used to validate their theoretical complexities and support.

➢ TASK 10 – Compare Theory with Experimental Behaviour

Theoretical analysis predicts an algorithm's efficiency using time complexity, while experimental analysis measures its actual execution time by running the program. Both methods help compare search algorithms for different dataset sizes.

➢ Theoretical Comparison

Algorithm	Average Time Complexity	Explanation
Linear Search	$O(n)$	Checks elements one by one.
Binary Search	$O(\log n)$	Divides the sorted list into halves.
Hash Table Search	$O(1)$	Finds records using a key directly on average.

➢ Experimental Analysis – Python Programs

A. Linear Search

This program checks each element until the key is found.

```python
import time

a = list(range(1000))
key = 999
start = time.perf_counter()

for i in range(len(a)):
    if a[i] == key:
        break

print("Index:", i)
print("Time:", round(time.perf_counter()-start, 6))
```

```
Index: 999
Time: 0.000139
```

B. Binary Search

This program searches a sorted list by repeatedly dividing the search range into halves.

```python
import time

a = list(range(1000))
key = 999
low, high = 0, len(a)-1
start = time.perf_counter()

while low <= high:
    mid = (low+high)//2
    if a[mid] == key:
        break
    elif a[mid] < key:
        low = mid+1
    else:
        high = mid-1

print("Index:", mid)
print("Time:", round(time.perf_counter()-start, 6))
```

```
Index: 999
Time: 3e-05
```

C. Hash Table Search

This program uses a Python dictionary to retrieve a record by its key.

```python
import time

students = {i: i for i in range(1000)}
start = time.perf_counter()

print("Student:", students.get(999, "Not Found"))
print("Time:", round(time.perf_counter()-start, 6))
```

```
Student: 999
Time: 3.9e-05
```

➢ **Comparison and Discussion**

Theoretical analysis shows that Linear Search takes $O(n)$, Binary Search takes $O(\log n)$, and Hash Table Search takes average $O(1)$time. Experimental results may differ because of hardware speed, system load, and measurement overhead.

Conclusion: Hash Table Search is suitable for frequent student record retrieval because it offers average constant-time lookup. Binary Search is also efficient when records are sorted, while Linear Search becomes slower as the dataset increases.

➢ **TASK 11 – Recommend and Defend the Final Design**

The student record system contains a large number of student records identified by unique Registration Numbers. Based on the theoretical analysis and experimental results of Linear Search, Binary Search, and Hash Table Search, the most suitable algorithm must be selected for efficient searching.

➢ **Recommended Algorithm**

Hash Table Search is recommended because it provides an average time complexity of $O(1)$. It retrieves student records directly using their Registration Numbers without checking every record individually.

➢ **Justification**

- Fast searching: Retrieves student records in constant average time.
- Large datasets: Suitable for handling 1,000,000 or more student records.
- Frequent requests: Efficient for handling 50,000 search requests per hour.
- Low update frequency: Since new records are added only once per day, the hash table can be updated when new records are received.
- Memory usage: Requires additional memory to store the hash table.
- Worst-case performance: Searching may take $O(n)$time if many keys experience collisions.

➢ **Python Implementation**

```
students = {
    101: "Arun",
    102: "Priya",
    103: "Ravi",
    104: "Kumar"
}

reg_no = 102

if reg_no in students:
    print("Student Name:", students[reg_no])
else:
    print("Student Not Found")
```

```
Student Name: Priya
```

Explanation: The Registration Number is used as the dictionary key. The program retrieves the corresponding student name directly if the key exists.

- **Comparison of Algorithms**

Algorithm	Average Complexity	Suitability
Linear Search	$O(n)$	Less suitable for frequent searches in large datasets
Binary Search	$O(\log n)$	Suitable when records are sorted
Hash Table Search	$O(1)$	Best suited for frequent exact-key searches

- **Final Conclusion**

Hash Table Search is the recommended solution because it provides fast average lookup, handles large datasets efficiently, and suits the system's low record-update frequency. Although it requires additional memory and can have slower worst-case performance, it is the most appropriate choice for frequent searches by unique Registration Number. Actual experimental timings should be used alongside theoretical complexity to validate this recommendation.

- **TASK 12 – Re-evaluate and Adapt the Design**

- **Problem Analysis**

The university student record system must handle 50,000 search requests per hour using a database containing 1,000,000 student records. New student records are added only once per day.

The main objective is to select an efficient search strategy that provides fast responses, handles a large number of requests, and minimizes unnecessary preprocessing.

- **Selection of Algorithm**

Hash Table Search is selected as the primary search strategy because it provides average $O(1)$lookup time using a student's unique Registration Number.

The hash table can be created from the existing student records and updated once daily when new records are added.

- **Proposed Design**

The system will use a hash table to store Registration Numbers as keys and student details as values.

1. Initial preprocessing: Load the one million student records into the hash table.
2. Search operation: Use the Registration Number to retrieve the required student record.
3. Daily updates: Add new student records to the hash table once per day.
4. Validation: Check whether the Registration Number exists and display the record or a “Student Not Found” message.
5. Performance monitoring: Measure search time and test the system with large datasets.

➢ **Time Complexity Analysis**

Operation	Time Complexity
Build hash table	$O(n)$average
Search a record	$O(1)$average
Insert a new record	$O(1)$average
Worst-case search	$O(n)$

Here, $n$represents the number of student records. The initial construction takes linear time, but this preprocessing cost is spread across many subsequent searches.

➢ **Python Implementation**

```
students = {
    101: "Arun",
    102: "Priya",
    103: "Ravi"
}

reg_no = 102

if reg_no in students:
    print("Student Found:", students[reg_no])
else:
    print("Student Not Found")
```

```
Student Found: Priya
```

This code demonstrates the basic search operation. In the actual system, the hash table would contain all one million records.

➢ **Evaluation of the Design Challenge**

The system receives 50,000 requests per hour, equivalent to approximately 14 requests per second on average. Hash Table Search is a suitable choice because its average lookup time does not grow linearly with the number of records.

However, actual performance depends on the implementation, memory availability, concurrent requests, and collision handling. Therefore, the complete system must be tested with one million records and a realistic request load before deployment.

➢ **Final Decision**

Decision: Retain Hash Table Search as the primary algorithm.

The existing strategy does not need to be replaced because hash-based lookup is well suited to frequent searches by unique Registration Number. Daily updates can be incorporated without rebuilding the entire table, provided the implementation supports efficient insertion.

Binary Search can be maintained as an alternative when ordered searching is required. The final

implementation should also include duplicate-registration checks, missing-record handling, and performance monitoring.

- **Conclusion**

Hash Table Search is the most appropriate strategy for this design challenge because it offers average $O(1)$lookup, supports frequent requests, and accommodates daily record updates. Testing under the expected workload is necessary to confirm that the system meets its response-time and resource requirements.